

程序设计

讲义

2026 年秋季

毕朝日

复旦大学集成电路与微纳电子创新学院

`zhaori_bi@fudan.edu.cn` `https://bzs2915.github.io/bzs2915/`

目录

第一讲 引言	4
1.1 什么是程序	4
1.1.1 0 和 1 的问题	5
1.2 机器如何做加法：从二进制到 ALU	5
1.2.1 二进制与逻辑代数	5
1.2.2 半加器	6
1.2.3 开关器件：继电器、电子管与晶体管	6
1.2.4 从晶体管到逻辑门	7
1.2.5 多位加法器与 ALU	7
1.3 存储程序计算机与指令执行	8
1.3.1 ENIAC 的问题：程序即接线	8
1.3.2 冯·诺依曼结构	8
1.3.3 一条指令长什么样	9
1.3.4 指令周期	10
1.4 从高级语言到机器码：编译器	10
1.4.1 从一行 C 代码到机器码	10
1.4.2 名字、地址与硬件	11
1.4.3 编译器的诞生	12
1.5 编程语言概观	12
1.5.1 定义与分类	12
1.5.2 编译型、解释型与中间语言	13
1.5.3 发展脉络，以及 C/C++ 与 Python 的分工	13
1.5.4 混合编程与两难问题	14
1.6 C 语言的来龙去脉	15
1.6.1 从 Multics 到 UNIX	15
1.6.2 大事年表	16
1.6.3 C 语言的优点	16
1.6.4 局限与修补	17
1.7 编程实例：最大公约数	17

1.8	使用 C 编程的七个步骤	18
1.8.1	从源程序到可执行文件	19
1.9	第一个 C 程序的结构	19
1.9.1	预处理指令与头文件	20
1.9.2	注释	20
1.9.3	函数与 main	21
1.9.4	变量与声明	21
1.9.5	标识符、关键字与保留标识符	22
1.9.6	printf 与格式化输出	23
1.9.7	return 语句	23
1.9.8	典型的 C 函数结构	23
1.9.9	增强程序的可读性	23
1.9.10	第二个例子	24
1.10	两类错误：语法错误与语义错误	24
1.10.1	语法错误	24
1.10.2	语义错误	25
1.10.3	第三种情况：代码之外	26
1.10.4	改错练习	26
1.11	本讲总结	28
	参考文献	28

第一讲 引言

Programs must be written for people to read, and only incidentally for machines to execute.

— 阿贝尔森与萨斯曼 (Abelson & Sussman, 《计算机程序的构造和解释》)

1.1 什么是程序

程序 = 让计算机完成任务的指令序列。

程序这个词在日常生活里就有。煮一碗方便面：烧开水，放入面饼，加调料包，等 3 分钟，关火。这套步骤有四个特征：步骤明确；顺序执行；每一步操作简单；组合后产生结果。计算机程序完全一样：由一系列简单指令组成，计算机逐条执行，最终完成复杂任务。单条指令能做的事很小，大量指令按确定的顺序组织起来，就能完成很大的工作。

程序的本体是对计算过程的精确描述，机器只是执行者。计算机史上的第一份程序写成时，能运行它的机器还不存在。机器换了一代又一代，描述计算过程的方法学却一直在积累，这门课教的就是这门方法学的当代形态。

历史旁注

用打孔卡序列控制机器的装置可以追溯到 1804 年雅卡尔 (Jacquard) 的提花机，织造的花纹由打孔卡决定。英国数学家巴贝奇 (Charles Babbage, 1791–1871) 在 1837 年前后完成通用计算装置“分析机” (Analytical Engine) 的设计：设有存放数据的仓库 (store) 与执行运算的作坊 (mill)，指令与数据均以打孔卡输入。因工程规模超出当时的制造能力、经费支持于 1842 年被英国政府终止，该机器未能建成。1842 至 1843 年，诗人拜伦的女儿洛夫莱丝 (Ada Lovelace, 1815–1852) 在梅纳布雷亚介绍分析机论文的英译本中撰写了七条注释，以 A 至 G 命名。注释 G 给出计算伯努利数的完整操作表，被计算机史学界视为人类写下的第一个计算机程序。她还预言，此类机器可对任何可编码的对象进行运算。1980 年，美国国防部将其主持设计的编程语言命名为 Ada，以纪念她。

1.1.1 0 和 1 的问题

这个定义还缺一半。方便面的步骤写给谁都能看懂，因为执行者是人；计算机程序的执行者是机器，而计算机只能理解 0 和 1。“烧开水”在机器层面没有对应物，每条指令都必须表示成 0 和 1 的序列。

1.2 机器如何做加法：从二进制到 ALU

1.2.1 二进制与逻辑代数

二进制算术由莱布尼茨（Leibniz）于 1679 年在手稿中建立，1703 年正式发表论文《论二进制算术》。1701 年，在华耶稣会传教士白晋（Bouvet）向他寄送伏羲六十四卦图，莱布尼茨注意到卦象的阴爻阳爻排列与 0 至 63 的二进制编码一一对应，并在论文中讨论了这一对应。二进制在计算机中被广泛采用，除了数学上的简洁性，核心原因是工程实现的可靠性：**物理器件更容易稳定地区分两种状态**。有电压与无电压、磁化的两个方向、有孔与无孔、导通与关断，都是容易获得的稳定对偶状态；十进制要求器件稳定区分十种状态，实现难度显著更高。

仅有两种状态的表示尚不足以支撑推理与运算，还需要描述逻辑关系的代数。1854 年，布尔（Boole）出版《思维的规律》，将与（AND）、或（OR）、非（NOT）纳入代数运算体系。1937 年，香农（Shannon）在麻省理工学院的硕士论文中证明布尔代数与开关电路同构，任何逻辑运算都可以由开关的串并联实现，为数字电路提供了统一的设计方法。

计算机怎么做 $5 + 3 = 8$ ？三个数的机器内表示为

$$5 = 101_{(2)}, \quad 3 = 011_{(2)}, \quad 8 = 1000_{(2)}.$$

二进制加法逐位进行，逢二进一：

$$\begin{array}{r} \text{进位: } 1 \quad 1 \quad 0 \\ 1 \quad 0 \quad 1 \\ + \quad 0 \quad 1 \quad 1 \\ \hline 1 \quad 0 \quad 0 \quad 0 \end{array}$$

从最低位起： $1 + 1 = 10$ ，本位记 0 并产生进位；第二位 $0 + 1$ 加进位 1 得 10，本位记 0 再进位；第三位 $1 + 0$ 加进位 1 得 10；进位落入第四位。计算机中所有的计算最终都归结为这样的位运算：乘法可分解为加法与移位，除法可分解为减法与移位，浮点运算由整数运算组合实现。

1.2.2 半加器

单个二进制位的加法由半加器（half adder）完成，输入为两个加数 A 、 B ，输出为本位和 Sum 与进位 Carry。

表 1.1: 半加器真值表

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

真值表对应的布尔表达式为

$$\text{Sum} = A \oplus B, \quad \text{Carry} = A \wedge B.$$

Sum 取异或运算，两个输入相同则结果为 0，不同则为 1；Carry 取与运算，两个输入都是 1 时结果才为 1。电路由两个逻辑门构成：异或（XOR）门输出 Sum，与（AND）门输出 Carry。布尔在 1854 年定义的运算，经香农 1938 年指明的路径实现为电路，逻辑代数至此具备了执行算术的能力。

1.2.3 开关器件：继电器、电子管与晶体管

逻辑门靠开关实现，开关用什么做，决定机器的运算速度。继电器、电子管、晶体管三代器件的更替，界定了早期计算机的发展阶段。

继电器以电流吸合金属触点，开关一次需克服机械惯性，速度限于每秒数次至数十次。1941 年祖思（Zuse）在柏林建成的 Z3、1944 年 IBM 与艾肯（Aiken）合作研制的哈佛 Mark I，均以继电器构成。

电子管以真空中电子的运动实现通断，无机械触点，速度提高约三个数量级。1945 年建成的 ENIAC 使用 17468 只电子管，每秒可完成约 5000 次加法；耗电 150 千瓦，电子管寿命有限，故障维护构成主要运行成本。第 1.3 节会再次遇到这台机器。

晶体管以半导体导电状态的变化实现通断，体积、功耗与寿命均优于前两代，速度持续提升。1947 年 12 月，贝尔实验室的巴丁（Bardeen）与布拉顿（Brattain）制成点接触晶体管，肖克利（Shockley）随后建立结型晶体管理论，三人获 1956 年诺贝尔物理学奖。制造方式的突破紧随其后：1958 年 9 月，德州仪器的基尔比（Kilby）在锗片上制成集成电路；1959 年，仙童半导体的诺伊斯（Noyce）以硅平面工艺实现可批量生产的集成电路。晶体管从此可以大规模集成于单枚芯片。基尔比因该工作获 2000 年诺贝尔物理学奖。

表 1.2: 开关器件的代际更替

机器	年代	开关器件	注记
Z3（柏林）	1941	电磁继电器	程序存放在打孔胶片上
哈佛 Mark I	1944	继电器	IBM 与艾肯合作，程序在纸带上
ENIAC	1945	17468 只电子管	编程靠插线，每秒约 5000 次加法
IBM 7090	1959	晶体管	晶体管商用计算机的代表
Intel 4004	1971	单芯片 2300 只晶体管	微处理器由此诞生

表中相邻两行的间隔对应摩尔（Moore）1965 年的观察：集成电路上的元件数约每年翻一番；1975 年他修正为每两年翻一番；英特尔高管戴维·豪斯随后提出“每 18 个月”的通俗版本。课件采用的表述为**晶体管密度每 18 至 24 个月翻一番**。该经验规律不具有物理定律的地位，性质上接近整个产业共同执行的发展日程，本专业后续课程，尤其是 EDA 方向，会反复引用它。

1.2.4 从晶体管到逻辑门

现代芯片采用 CMOS 工艺实现逻辑门。以 CMOS 反相器（NOT 门）为例：上管为 PMOS，接电源 V_{DD} ；下管为 NMOS，接地 GND。输入为 0 时 PMOS 导通、NMOS 关断，输出为 1；输入为 1 时 PMOS 关断、NMOS 导通，输出为 0。反相器由两只晶体管构成，与门、或门、异或门由若干晶体管组合实现。

晶体管的物理实体为 MOSFET 结构：在硅基底上制作源（S）、漏（D）两极，中间隔氧化层，栅（Gate）位于氧化层上方，栅极电压控制源漏之间的导通。几个关键参数：现代工艺特征尺寸为 3nm 至 7nm；单芯片集成数十亿晶体管；开关频率达 GHz 量级。对照表 1.2 中每秒数步的继电器，百余年间开关速度提升超过八个数量级。

1.2.5 多位加法器与 ALU

半加器不处理低位进位。带低位进位输入的加法单元称为全加器（full adder, FA），串联 N 个全加器构成 N 位加法器： $A[3:0] + B[3:0] = S[3:0]$ 由四个全加器串联实现，32 位或 64 位加法器即 32 个或 64 个全加器串联，进位从低位传向高位。

这种结构存在**进位链延迟**：最高位的运算须等待全部低位进位逐级传播，位数越多延迟越大。超前进位加法器以额外的电路规模直接由输入计算各位进位，避免逐级等待，是以电路规模换取运算速度的典型设计。

加法之外，把实现逻辑、比较、移位功能的电路与加法器集成，并引入控制信号，即构成 **ALU**（算术逻辑单元，Arithmetic Logic Unit）。ALU 的输入为操作数 A、B 与控制信号，输出为运算结果，可执行四类操作：

- 算术：加、减、乘、除；
- 逻辑：AND、OR、NOT、XOR；
- 比较：<、>、=；
- 移位：左移、右移。

控制信号决定执行的运算。“由控制信号选择操作”是指令概念的雏形：一条指令，即一串规定“执行哪种操作、作用于哪些数据”的 0 和 1 序列。

1.3 存储程序计算机与指令执行

1.3.1 ENIAC 的问题：程序即接线

1945 年建成的 ENIAC 是当时运算速度最快的计算装置，其“编程”方式为物理接线。ENIAC 没有存储程序的部件，计算新问题时须先在纸面上将问题分解为机器部件的连接方案，再在机器上插接数百根电缆、设置数千个开关，耗时以天计。机器的运算速度为每秒约 5000 次加法，而程序更换的时间以天计，二者相差多个数量级。承担该项工作的是六位程序员，为首的是让·巴蒂克（Jean Bartik）。她们是人类最早的程序设计人员。

问题的本质在于：程序不存在于存储器中，它表现为机器的物理接线状态，修改程序即修改机器。程序与数据分离的结构，限制了计算机能力的发挥。

1.3.2 冯·诺依曼结构

1944 年，数学家冯·诺依曼（von Neumann）经由戈德斯坦（Goldstine）加入 ENIAC 的后续项目 EDVAC，1945 年 6 月执笔完成 101 页的《EDVAC 报告书的第一份草案》（First Draft of a Report on the EDVAC）。该报告仅署名冯·诺依曼一人，最初合作者埃克特与莫奇利对此持保留意见；报告提出的方案可概括为一句：

将指令作为数据，与数据共同存入同一个存储器。

由此，取指令与取数据使用同一套电路。修改程序不再涉及任何物理接线，只需改写存储器中的 0 和 1 序列；程序成为可以加载、修改、复制、保存的数据对象。软件开发作为一种独立活动的技术前提由此确立。

采用该方案的计算机称为**存储程序计算机**（stored-program computer），习惯上称冯·诺依曼结构，由五个部件组成：

- CPU：内含控制单元、ALU 和寄存器；

- **存储器**：同时存放指令和数据；
- **输入设备与输出设备**；
- 各部件之间由**总线**连接。

三个关键特征：

1. **存储程序**：指令和数据都存在存储器中，指令本身也是 0 和 1；
2. **顺序执行**：按地址顺序执行指令，除非遇到跳转；
3. **二进制**：所有信息都用二进制表示。

CPU 的工作流程为循环：取指令，译码，执行，存储结果，再取下一条指令。该循环自 1945 年沿用至今。

1.3.3 一条指令长什么样

一条指令长什么样？设想一台最简的机器：CPU 里只有一个寄存器，叫累加器（accumulator）。它把存储器中的两个数相加、结果写回存储器，需要三条汇编指令：

```
LOAD [b]
ADD  [c]
STORE [a]
```

设存储器中 $b = 5$ 、 $c = 3$ 。执行过程：LOAD 把 b 的值 5 从存储器读入累加器；ADD 把 c 的值 3 与累加器中的 5 相加，和 8 留在累加器；STORE 把 8 写回存储器中的 a 。执行结束时，存储器中 $a = 8$ 。

每条指令的动作：

- **LOAD**：从存储器读数据到累加器；
- **ADD**：存储器的数与累加器相加；
- **STORE**：累加器写回存储器。

这个例子给出了 CPU 的基本工作方式：**数据必须先进寄存器；ALU 只能操作寄存器；结果再存回存储器**。寄存器是 CPU 内部容量小、速度高的存储单元，ALU 的输入输出直接连在寄存器上，没有直通存储器的通道。本例的模型只设一个寄存器，即累加器；真实的 CPU 配有一批通用寄存器，指令中须写明寄存器编号，例如 LOAD R1, [b]，工作方式相同。一次加法须经读、算、写三个动作，这就是“一行高级语言代码对应多条机器指令”的根源。

1.3.4 指令周期

CPU 怎么一条条执行它们？程序在存储器中按地址存放：

地址	指令
100	LOAD [b]
104	ADD [c]
108	STORE [a]

CPU 内设专用寄存器程序计数器（PC，Program Counter），存放当前指令的地址。上例中 PC= 104 时，正在处理 ADD [c]，此时累加器中已装入 5。

每条指令的执行经过**指令周期**的四个阶段：

1. **取指（Fetch）**：从存储器读取下一条指令；
2. **译码（Decode）**：解析指令含义；
3. **执行（Execute）**：ALU 执行运算；
4. **写回（Write back）**：保存结果。

该循环每秒重复数十亿次，这就是主频 GHz 的物理含义：主频 3GHz 表示这样的节拍每秒重复三十亿次。

现代 CPU 以流水线技术使四个阶段重叠进行，并可能在一个节拍内完成多条指令，因此 GHz 计数的节拍数与每秒完成的指令数并不相等。但无论流水线深度如何，取指、译码、执行、写回的骨架自 1945 年延续至今。

1.4 从高级语言到机器码：编译器

1.4.1 从一行 C 代码到机器码

第1.3节的模型里，两个数相加要写三条指令，取数、运算、存回全靠人工逐条安排。程序员实际想写的只有一句：

```
int a = b + c;
```

编译器把它转换为三条汇编指令：

```
LOAD [b]
ADD [c]
STORE [a]
```

对应关系有三条：

- 一行 C 代码对应多条汇编指令；
- 变量名对应存储器地址；
- 运算表达式对应寄存器操作。

汇编语言再经**汇编器**翻译为机器码。在这台教学模型上，每条指令编码为 8 位二进制：前 4 位为操作码，后 4 位为内存地址。操作码是指令集中为每种操作规定的固定编号，机器见到约定编号即执行对应操作。编码映射为：操作码 LOAD= 1011、ADD= 0000、STORE= 1010；内存地址 $[a] = 0001$ 、 $[b] = 0010$ 、 $[c] = 0011$ 。三条指令对应的机器码见表1.3。

表 1.3: 从汇编到 8 位机器码

汇编指令	含义	机器码
LOAD $[b]$	将 b 读入累加器	1011 0010
ADD $[c]$	将 c 与累加器的值相加	0000 0011
STORE $[a]$	将累加器的结果存入 a	1010 0001

这就是 CPU 真正执行的内容。存储器中保存的就是这些 0 和 1；按地址顺序取指执行所取出的，同样是这些 0 和 1。机器语言无法阅读、无法修改、难以查错，直接用它编程并不现实。编程 = 用人类可读的方式描述计算，交由编译器翻译为机器指令。

1.4.2 名字、地址与硬件

编程的本质：把人类的**思维逻辑**，转化为计算机**能执行的指令**，中间以**编程语言**为载体。以“计算平均分”为例：想法先被表达为编程语言里的 $avg = sum/n$ ；再经编译成为机器码 10110010...。

程序员承担语义层面的转换：把意图分解为精确步骤，本课训练的就是这一能力。编译器承担机械层面的转换：翻译规则完全确定，适合由机器执行。

三条对应关系里，“变量名对应存储器地址”是 C 语言通向硬件的接口。变量的本质是一个被命名的存储单元，名字之下是地址。C 语言允许程序直接取得并持有地址，这种数据就是指针，后续章节详述；硬件外设的控制与状态寄存器同样映射在处理器的地址空间中，这种设计称为内存映射 I/O。因此 C 程序可以像访问变量一样访问硬件寄存器：嵌入式开发中对定时器、串口的操作，EDA 仿真器对设计数据的底层操作，走的都是这条通道。理解“名字-地址-内容”三个层次，后面学指针时用处直接。

1.4.3 编译器的诞生

“让机器完成程序翻译”在 1940 年代末曾被普遍认为不可行，当时的观念是程序设计只能由人完成，机器只负责执行。1952 年，美国海军的格蕾丝·霍珀（Grace Hopper）在 UNIVAC 上实现 A-0 编译器，把写好的子程序自动装配为完整程序¹。

使行业接受编译器的是 FORTRAN。1953 年，IBM 的约翰·巴科斯（John Backus）提交建议书，提议为科学计算开发“自动编程”系统；包括他本人在内，多数人预期编译产物会比手工汇编慢得多。1957 年编译器交付，编译出的数值代码效率接近手写汇编，效率疑虑随之消除，科学计算从此普遍采用高级语言。这段历史留下的判断标准至今有效：凡规则完全确定的翻译交给机器，人负责规则之外的语义。

编译器的功能有两个方向：

- 将**相同**的高级语言程序，转换为针对**不同**硬件的机器语言。同一份 C 源码，在 x86 与 ARM 平台上由各自的编译器生成对应的机器码，源码无须修改，这是可移植性的来源；
- 将**不同**的高级语言程序，转换为针对**同一台**机器的机器语言。C、C++、Fortran 程序在同一台机器上都变为该 CPU 的指令，这是语言与硬件解耦的来源。

一个只含 `printf("1+1=%d", 1+1);` 的程序，经编译链接后得到的是纯粹由 0 和 1 组成的文件，可执行文件中不含任何人类可读的内容。这是“存储程序”的直接体现。

1.5 编程语言概观

1.5.1 定义与分类

编程语言（Programming Language）用于定义计算机程序，具有确定的语法规则。“确定的语法规则”是它与自然语言的本质差别，第1.10节的错误分类会回到这一点。

编程语言沿三个维度分类。

按抽象级别：机器语言、汇编语言、高级语言。机器语言是 CPU 直接执行的 0 和 1 序列；汇编语言以 LOAD、ADD 这样的助记符替代 0 和 1 串；C、Python 等高级语言面向人的思维。第1.3节的 $a = b + c$ 例子恰好把三级串成一条链。

按实现方式：编译型、解释型、混合型（中间语言）。

按编程范式：命令式语言描述“怎么做”，逐步给出改变状态的动作，含面向过程的 C 与面向对象的 C++、Java。面向对象的思想源于 1967 年挪威计算中心的达尔

¹霍珀对此的回忆是“Nobody believed that I had a running compiler and nobody would touch it. They told me computers could only do arithmetic.”（没人相信我真有一个能运行的编译器，也没人愿意碰它。他们告诉我，计算机只会做算术。）

(Dahl) 与尼加德 (Nygaard) 在 Simula 67 中引入的类与对象；1979 年斯特劳斯特鲁普 (Stroustrup) 在 C 上添加类，1983 年定名 C++。声明式语言描述“要什么”。函数式的 Lisp 由麦卡锡设计，1958 年成形，1960 年发表正式论文；逻辑式的 Prolog 出自科尔默朗等人之手，1972 年诞生于马赛。另有定位通用型的 Ada、C++ 等。

1.5.2 编译型、解释型与中间语言

- 编译型 (C/C++)：源代码 (.c) 经编译器整体翻译为机器码，CPU 直接执行；
- 解释型 (Python)：源代码 (.py) 由解释器逐行读取、逐行执行。

两条路径的取舍见表1.4。编译型把翻译成本支付在运行之前，换取执行速度；解释型把翻译成本推入运行过程，换取灵活与跨平台。

表 1.4: 编译型与解释型语言对比

	编译型 (C/C++)	解释型 (Python)
执行速度	快	慢
运行环境	无需	需要解释器
源码保护	好	暴露
开发周期	长	短 (开发效率高)
跨平台	平台相关	容易
调试	较困难	方便

二分法是简化。工程实践中存在中间地带，即课件所说的**混合型 (中间语言)**。1995 年 Java 以“一次编写，到处运行”为目标，采用中间方案：编译器先把源码翻译为面向虚拟机的**字节码**，Java 虚拟机 (JVM) 逐条解释执行，并对热点代码即时编译为机器码。CPython 同样先把.py 文件编译为字节码再由虚拟机执行；浏览器中的 JavaScript 由 V8 引擎即时编译为机器码。因此更准确的描述是：编译与解释是一条连续谱，纯编译与纯解释位于两端，中间语言占据中段。表中“执行速度慢”也应据此理解：带即时编译的解释器在数值代码上可以接近编译型的速度。

1.5.3 发展脉络，以及 C/C++ 与 Python 的分工

编程语言七十余年的发展可归纳为以下节点。**Ada**，1843 年：程序的概念先于机器存在，经过见第1.1节的旁注。**FORTRAN**，1953 年立项，1957 年交付，第1.4节已述其经过：它使高级语言可用，定位于科学计算。名字是 FORmula TRANslation 的缩写，意思是公式翻译。**Lisp**，1958 年：麦卡锡为人工智能研究设计的函数式语言，“代码即数据”的思想至今保留在各函数式语言中。**结构化程序设计**：1968 年戴克斯特拉

(Dijkstra) 在《ACM 通讯》发表短信《Go To Statement Considered Harmful》，主张程序流程只用顺序、选择、循环三种结构表达，废除无限制跳转。该主张为全部主流语言接受，C 语言的 `if`、`while` 与函数即三种结构的具体化。**ALGOL 60**：1958 年的 ALGOL 58 经国际委员会修订为 ALGOL 60，报告由璁尔主编，语法用巴科斯发明的一套记法描述，这套记法后来称为巴科斯-璁尔范式，简称 BNF。ALGOL 60 的块结构与形式文法方法为后世语言继承。它的直系传承自剑桥 1963 年的 CPL 开始，经里查兹 1967 年的 BCPL 与汤普森 1969 年的 B，最后由里奇在 1973 年发展为 C。

就本专业而言，当下语言版图中关系最直接的是 C/C++ 与 Python 的组合。C/C++ 承担高性能计算与底层引擎：操作系统内核、嵌入式系统、EDA 工具的计算核心（综合、布局布线、仿真引擎）均以 C/C++ 实现。Python 承担高层抽象与自动化脚本：设计流程驱动、数据处理、测试激励生成大量使用 Python 完成，EDA 工具传统的脚本接口 Tcl 与之角色相近。Rust、Go、Java、JavaScript 等语言各覆盖系统编程、云原生、企业应用、Web 等领域，本课不展开。几种常见语言的特性对比见表 1.5。

表 1.5: 常见语言特性对比

特性	C/C++	Java	Python	JavaScript
执行速度	极快	快	慢	中等
开发效率	低	中等	高	高
内存控制	手动	自动 GC	自动 GC	自动 GC
学习曲线	陡峭	中等	平缓	平缓

1.5.4 混合编程与两难问题

编译型与解释型各有优势，工程实践发展出兼取两者的**混合编程**。动机有三：不同语言有不同的优势区间；性能与开发效率需要平衡；利用现有生态系统。

最典型的实例是 NumPy，用户层为 Python，性能核心为 C：

用户 Python 代码 → NumPy Python 层 → C/C++ 核心 → BLAS/LAPACK

用户使用 Python 层的 API：

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
c = np.dot(a, b)
```

`np.dot` 最终调用 C 实现的高性能例程：

```
double dot_product(double* a, double* b, int n) {  
    double sum = 0.0;  
    for (int i = 0; i < n; i++)  
        sum += a[i] * b[i];  
    return sum;  
}
```

最底层为 BLAS 与 LAPACK，数值计算领域持续维护四十余年的线性代数基础库。

科学计算领域将这一结构性矛盾称为**两难问题**（two-language problem）：数值原型适合用 Python 快速验证，性能要求又迫使同一逻辑以 C/C++ 重写，项目因此同时维护两种语言的两套代码。2012 年问世的 Julia 语言即以消除该两难为目标：语法接近 Python，编译器直接生成机器码。用 C/C++ 做引擎、用 Python 或 Tcl 做胶水的分层至今仍是工程主流，也是现代芯片设计与验证工作流的核心范式：仿真器、综合与布局布线引擎以 C/C++ 实现计算密集部分，流程驱动、结果处理与自动化以 Python 或 Tcl 完成。本课教授 C，对应引擎一侧；胶水一侧的 Python，各位在其他课程中已有接触。

1.6 C 语言的来龙去脉

1.6.1 从 Multics 到 UNIX

C 语言的起源与一项终止的工程相关。1964 年，MIT、通用电气与贝尔实验室联合启动 Multics 项目，目标是将计算机构造为多人共享的公用计算设施。系统规模持续膨胀而性能未达预期，1969 年 4 月贝尔实验室退出。

项目成员汤普森（Ken Thompson）与里奇（Dennis Ritchie）由此失去计算平台与项目。汤普森编写了一个模拟太阳系航行的太空游戏 Space Travel，它原运行于 Multics，此后移植到实验室的 GE 机器，机时费高昂；他随后将游戏移植到一台闲置的 PDP-7 小型机，并为其开发了一套软件工具：文件系统、进程管理、一组命令与编辑器。1970 年，柯林汉将该系统命名为 Unics，这个名字是对 Multics 的双关，后来定型为 UNIX。

开发这套系统时，汤普森在 BCPL 的基础上简化出 **B 语言**，BCPL 是里查兹 1967 年设计的语言；B 无类型、接近硬件，满足当时的需要。1970 年起项目迁移到 16 位字长的 PDP-11，B 按字寻址、解释执行的方式不适应新机型对字符处理与速度的要求。里奇在 1971 至 1973 年间对 B 进行改造：增加类型系统，解释改为编译，语言顺字母序定名 **C**。1973 年，UNIX 以 C 完全重写。在一台 24KB 内存的机器上发展出的系统与语言，此后成为整个行业的基础设施。

1.6.2 大事年表

- 1973 年，UNIX 第四版以 C 重写完成。操作系统自此可以整体以高级语言书写；
- 1978 年，柯林汉与里奇合著的 *The C Programming Language* 出版，即 K&R。书中首个示例程序输出 “hello, world”，此后成为编程语言教材的通行首例；
- 1983 年，汤普森与里奇获图灵奖，授奖词表彰他们 “发展了通用操作系统的理论，特别是 UNIX 操作系统的实现”；
- 1989 年，美国国家标准协会（ANSI）颁布 C 标准，即 C89，又称 ANSI C 或标准 C。标准使 C 未分裂为互不兼容的方言；
- 1999 年，ISO 颁布 C99 标准。此后有 2011 年的 C11、2018 年的 C17 以及最新的 C23，语言持续演进。

一个源于游戏移植项目的工具语言，此后成为操作系统、嵌入式系统、数据库和 EDA 工具的基础语言。

1.6.3 C 语言的优点

- **强大的控制结构：**结构化、模块化；
- **紧凑而高效：**程序执行速度快、内存占用小。C 的发展平台是一台 24KB 内存的机器，其设计目标包含替代汇编语言，须在有限空间内高速运行；
- **可移植性：**运行于多种系统与机型；C 标准的制定避免了语言分裂为互不兼容的分支；编译器规模小、易于编写，便于移植到新机器；
- **强大而灵活：**可用于实现其他语言（Perl、Python、FORTRAN 等）的编译器或解释器，可用于开发操作系统（UNIX），可以直接访问内存与硬件，靠的正是第1.4节讲的名字与地址映射。这份灵活而危险，“危险”在哪里要等学了指针与内存布局才说得清，此处先记下；
- **与 UNIX 和 Linux 的结合：**两者的历史深度耦合；
- **介于高级语言和低级语言之间：**具备高级语言的控制结构，同时可像低级语言一样直接访问内存与硬件；
- **学习其他语言的基础：**C++、Java、Python 的语法均从 C 继承了相当多成分。

1.6.4 局限与修补

缺少高级抽象。C 没有内置字符串类型，没有泛型编程支持，没有异常处理机制，内存由程序员手动管理，容易出错。

开发效率偏低。实现相同功能需要更多代码；缺少现代标准库；与内存相关的调试困难。以字符串拼接为例。C 写法：

```
char str1[100] = "Hello";
char str2[] = "World";
strcat(str1, str2); /* append str2 to the end of str1 */
```

str1 和 str2 都是字符数组，C 没有现成的“字符串相加”，要调用库函数 strcat 逐字符复制，而且要程序员自己保证 str1 预留的空间装得下两段文字。Python 对比：

```
s = "Hello" + "World"
```

一行完成，空间分配由语言自动处理。差别背后是两种设计取向：Python 替程序员管理内存；C 把内存交给程序员，代码紧凑、执行快，空间是否充足由程序员预先考虑。空间不足时会发生什么，涉及内存的布局方式，留到指针一章再展开。C 的设计哲学是假定程序员了解自己在做什么：控制权与出错后果均由程序员承担。

后续标准的修补。C99/C11/C17/C23 陆续加入变长数组（VLA）、_Bool 类型与 <stdbool.h>、_Static_assert 静态断言、_Generic 泛型选择、原子操作和线程支持。C 的基本设计取向保持不变，依旧是贴近硬件、信任程序员。

1.7 编程实例：最大公约数

求两个自然数的最大公约数。这个问题的解法是现存最古老的算法之一，通称欧几里得算法；“算法”（algorithm）一词源自九世纪数学家花拉子米（al-Khwarizmi）著作标题的拉丁化。欧几里得《几何原本》（约公元前 300 年）卷七记载的正是本例：**不断从较大数中减去较小数，直至两数相等，结果即两数的最大公约数。**课件采用的减法表述即欧几里得的原版。

手算 $GCD(16, 24)$ ：

$$\begin{aligned} GCD(16, 24) &= GCD(16, 24 - 16) = GCD(16, 8) \\ &= GCD(16 - 8, 8) = GCD(8, 8) = 8. \end{aligned}$$

流程为：输入 A、B；比较 A 与 B：若 $A < B$ ，则 $B \leftarrow B - A$ ；若 $A > B$ ，则 $A \leftarrow A - B$ ；若 $A = B$ ，输出 A，结束。比较、减法、循环，分别对应 ALU 的比较运算与程序的选择、循环结构。现代教材常用取余版本 $GCD(a, b) = GCD(b, a \bmod b)$ ，它

与减法版逻辑内核一致，差异在收敛速度：拉梅（Lamé）于 1844 年证明取余版步数与斐波那契数相关，不超过较小数十进制位数的 5 倍。

C/C++ 实现：

```
/* GCD in C */
while (a != b) {
    if (a > b) a -= b;
    else      b -= a;
}
```

Python 实现：

```
# GCD in Python
while a != b:
    if a > b:
        a -= b
    else:
        b -= a
```

同一算法在两种语言中的表述逐词对应，差异仅在形式：C 的语句以分号结尾、以花括号组织代码块；Python 以缩进表达相同结构。算法与语言无关，语言是承载算法的记法。

1.8 使用 C 编程的七个步骤

从问题到可交付程序，完整的路径包含七步：

1. **定义程序目标。**回答两个问题：程序的输入是什么？程序应完成什么功能、输出什么？此阶段不涉及具体的语言和代码；
2. **设计程序。**确定数据的表示方法与完成功能的步骤，前者是数据结构，后者是算法。沃斯（Wirth）1976 年的著作以书名给出该等式：*Algorithms + Data Structures = Programs*。把具体问题抽象成数据结构与算法的工作，主要发生在这两步；
3. **编写代码。**借助集成开发环境完成，本课使用 VS Code，覆盖代码编写、编译/链接、调试/运行；
4. **编译；**
5. **运行。**例如 Windows 系统下运行生成的.exe 文件；
6. **调试与测试。**debug；

7. **维护和修改。**增加新功能，采用新算法提高程序效率。

前两步的时间投入往往超过后面五步的总和。常见的失误是跳过前两步直接编码：目标定义不完整，程序在逻辑正确的前提下仍可能答非所问；数据结构选择不当，后续算法的性能无从谈起。

1.8.1 从源程序到可执行文件

C 程序从源代码到可执行文件经过四个阶段：

表 1.6: 从源代码到可执行文件

阶段	工具	说明
源代码	编辑器 或 IDE: VS Code、Vim、Sublime、Notepad++	程序员书写的文本文件
↓ 编译器	gcc / clang	将高级语言转换为目标文件（.o 文件）
↓ 链接程序	构建系统: CMake、Makefile	目标代码与库代码、启动代码合并。启动代码是程序与操作系统的接口
↓ 可执行程序		生成可执行代码（二进制文件）

表中“编译器”一栏实际包含预处理、编译、汇编多道工序，gcc 等工具链按此顺序执行。链接器的输入有三部分：目标代码、库代码与启动代码。库代码是 printf 这类函数的实现；启动代码完成 C 运行环境的初始化，随后进入 main 函数。缺少任一部分都无法完成链接。链接产物为二进制文件，内容只有机器码。

1.9 第一个 C 程序的结构

一个完整的 C 程序：

```
#include <stdio.h>
/* a simple C program */
int main(void) {
    int num;
    num = 1;
```

```
printf("I am a good ");  
printf("computer.\n");  
printf("%d is my favorite number.\n", num);  
return 0;  
}
```

运行输出：

```
I am a good computer.  
1 is my favorite number.
```

自然语言容许部分错误，个别错别字与语序问题通常不妨碍理解；计算机语言依赖编译器，格式必须完全正确、符合规则，一个分号的缺失就足以让编译器拒绝整个程序，第1.10节的改错练习就是一例。

1.9.1 预处理指令与头文件

`#include <stdio.h>` 是 C 的预处理语句，在编译之前执行：把 `stdio.h` 的全部内容复制到这一位置。

`stdio.h` 即 standard input/output header，标准输入输出头文件，包含 `printf()` 等函数的信息。头文件以 `.h` 为后缀，包含常量定义、函数声明等，为编译链接提供信息。需要用到的头文件必须写出，漏写头文件是初学者常见的编译错误来源。

1.9.2 注释

`/* a simple C program */` 是注释。注释用于提高程序的可读性，可置于程序的任意位置。`/* */` 可用于单行与多行；`//` 用于单行注释。注释可使用中文或英文，课件示例为中文注释 `/* 一个简单的 C 程序 */`。

工程惯例是在文件开头以注释块说明程序名、目的与作者：

```
/* Name:      pun.c  
   Purpose: Prints a bad pun.  
   Author:   K.N.Dominic */
```

关于注释有三点需要注意：

- 编译器以一个空格替代注释。`a/* 注释 */b` 在编译器处理结果中为 `a b`，注释不会使两个标识符粘连；
- `/* */` 的嵌套不合法。注释在 `*/` 处结束，多余的 `*/` 报错。下例中 `/*` 与注释体中的 `*/` 配对，末行的 `*/` 失去配对对象：

```
/*
    *** wrong ***
*/
```

- 标识符长度有限。C89 标准只要求编译器区分前 **31** 位，C99 将下限提高至 63 位。超出该长度的两个仅尾部不同的名字可能无法区分。

1.9.3 函数与 main

`int main(void)` 中各成分的含义：圆括号表明 **main 是一个函数**；`int` 表示该函数返回一个整数；`void` 表示该函数不接受任何参数。

main 函数的特殊地位：

- 所有 C 程序以 `main` 函数为入口；
- `main` 函数的返回值返回给操作系统；
- 标准形式为 `int main(...)`；也存在 `void main(...)` 的写法，不建议使用。

一般地，**C 程序是函数的集合**。函数是组合在一起的语句，完成一个子功能，拥有名字，对应模块、过程或子进程的概念。函数分两类：一类由程序员自行编写；另一类是 C 标准库函数，由编译器或平台提供，`printf` 属于此类。一个 C 程序可含多个函数，`main` 函数为必需。

花括号 `{ }` 标记**函数体**（程序体）的开始与结束，并用于组织代码块。

1.9.4 变量与声明

`int num;` 是**变量声明**，它同时声明了变量的数据类型 `int` 与变量名 `num`。声明格式为：类型、空格、变量名。

编译器依据该语句为 `num` 分配存储空间，空间大小由类型决定，空间的名称即变量名。变量声明在源代码层面建立了名字与类型的绑定，生成目标代码时编译器把名字落实为具体的存储地址，这正是第1.4节讲过的名字与地址的映射。`num` 这类名字称为**标识符**，即变量或函数的名字。

C 要求**所有变量使用之前必须定义/声明**；C99 及后续标准允许在函数体内交错书写声明与语句，也可以把变量集中声明在函数体开头。多个变量可合并于一条声明：`int a, b, c;`，等价于 `int a; int b; int c;`。`int a.b.c` 为错误写法，分隔符应使用逗号。末尾分号表明这是一条 C 语句。

集中声明便于查看函数用到的变量；在第一次使用变量附近声明并初始化，则便于结合计算过程理解用途。两种位置安排都要满足先声明、后使用的要求。

数据类型回答“C 语言可处理哪些数据”。基本数据类型有三类：**整型**对应整数，**浮点型**对应实数，**字符型**对应字符。本例的 `int` 为整型。

`num = 1;` 是**赋值语句**：把 1 赋给变量 `num`。`num` 此后可被赋予不同的值。

1.9.5 标识符、关键字与保留标识符

变量、符号常量、函数等的命名规则：

- 使用有意义的名字，必要时以注释解释；
- 长度与编译器和系统有关，存在上限，C89 保证区分前 31 位；
- 可用字符为**大小写字母、数字、下划线**；不能使用其他符号；不能以数字开头；
- 操作系统与 C 库的标识符以下划线开头，用户程序应避免以下划线开头命名；
- **大小写敏感**：`num` 与 `Num` 是两个标识符；
- 避免使用关键字与保留标识符。

推荐的命名风格有两种：`current_work`、`name_address` 这样的下划线连接式，以及 `currentWork`、`nameAddress` 这样的驼峰式。选定一种并在整个程序中保持一致。

关键字为语言保留的词，大小写固定。`int` 和 `return` 写成小写，`_Bool` 则含大写 B；不能任意改变拼写。数据类型关键字如 `int`、`double`、`struct`、`static`、`const`、`extern` 等；结构关键字如 `if`、`for`、`switch`、`continue`、`goto`、`return`、`typedef` 等。保留标识符虽然不是关键字，同样应避免：预处理标识符如 `define`、`ifdef`、`include`，标准库函数名如 `printf`、`scanf`。

表1.7给出九个名字的合法性判断。

表 1.7: 标识符合法性判断

标识符	判断	理由
<code>a</code>	合法	
<code>#a</code>	不合法	<code>#</code> 不是可用字符
<code>_A</code>	合法但应避免	以下划线开头，与系统库命名冲突风险大
<code>1A</code>	不合法	不能以数字开头
<code>c1_2</code>	合法	字母、数字、下划线的组合
<code>>c1</code>	不合法	<code>></code> 不是可用字符
<code>my_name</code>	合法	
<code>;my_name</code>	不合法	分号不是可用字符
<code>hot-weather</code>	不合法	连字符会被当作减号

1.9.6 printf 与格式化输出

`printf()` 是 C 的标准输出函数，该名字继承自 B 语言的函数库。括号表明这是函数；括号中的内容是从 `main` 传递到 `printf` 的**实际参数**；`printf` 的功能是打印实际参数；最外层双引号不输出。

`\n` 表示换行，使后续输出从下一行开始；`\r` 表示回车，使输出位置回到本行开头。回车与换行是不同的控制字符。本程序前两个 `printf` 合成第一行输出，第三个 `printf` 输出第二行。

`%d` 中，`%` 为**占位符**，表示该位置替换为变量的值；`d` 表示以十进制整数形式输出，这一机制称为**格式化输出**。`%d` 位于双引号内，对应变量的值 `num` 作为参数跟在逗号之后，运行时 `%d` 的位置填入 `num` 的值 1。

1.9.7 return 语句

`return 0`；给出 **main 函数的返回值**。返回值为整型；0 表示程序正常结束。漏写 `return` 时程序可能仍能编译，只是会伴随警告；惯例要求写出，以避免错误。

该返回值交付操作系统：UNIX shell 中以 `$?` 查询上一个程序的返回值，Windows 命令行对应 `%ERRORLEVEL%`。脚本与构建工具依据该值判断程序是否正常结束，`Makefile` 就是一例。程序从入口到退出始终与操作系统交互。

1.9.8 典型的 C 函数结构

本程序的骨架即典型的 C 函数（程序）结构：

- 预处理指令 (`#include <stdio.h>`)；
- 函数头 `int main(void)`，`main` 为函数名；
- 函数体（花括号内），依次包含：变量声明 (`int num;`)、赋值语句 (`num = 1;`)、函数调用 (`printf(...)`)、`return 0;`。

1.9.9 增强程序的可读性

同一程序的不同写法在可读性上差异显著。可读性清单：

- 选择有意义的标识符；
- 使用注释；程序开头以注释说明该函数/程序的目的等；
- 使用空行进行概念分隔；
- 每个语句一行，过长语句可分成多行；

- 缩进;
- 在合适位置添加空格, 如逗号、运算符之后。

程序首先是写给人读的, 这份清单就是“写给人读”的具体做法。

1.9.10 第二个例子

多变量的例子:

```
#include <stdio.h>

int main(void) {
    int feet, fathoms;

    fathoms = 2;
    feet = 6 * fathoms;
    printf("There are %d feet in %d fathoms!\n", feet, fathoms);
    printf("Yes, I said %d feet! \n", 6 * fathoms);
    return 0;
}
```

输出两行: There are 12 feet in 2 fathoms! 与 Yes, I said 12 feet!。1 英寻合 6 英尺, 6 乘 2 得 12。

这个例子里还有两处细节。int feet, fathoms; 一条声明定义两个变量, 共用类型 int。printf 的参数可以是表达式: 第二个 printf 的参数 6 * fathoms 在传参之前完成求值, 占位符%d 填入表达式的结果 12。

1.10 两类错误: 语法错误与语义错误

错误 (bug) 一词在工程语境中的使用可追溯到 1878 年爱迪生的书信。1947 年 9 月 9 日, 哈佛 Mark II 计算机的一只继电器触点间发现一只飞蛾, 霍珀的团队成员将其贴入工作日志并注明“第一个被实际发现的 bug”, 该日志现存华盛顿史密森学会。此后查错、除错的过程称为 debugging。

1.10.1 语法错误

语法错误指违反 C 语言格式规则的错误, 由编译器检查。以下代码包含典型的语法错误:

```
#include <stdio.h>
```

```

int main(void) (          /* ( should be { */
    int n, int n2, int n3; /* should be int n, n2, n3; */
    /*the program has a syntax error

    n = 5;
    n2 = n*n;
    n3 = n2*n2;
    printf("n=%d, n square = %d, n cubed = %d\n", n, n2, n3)

    return 0;

)                          /* ) should be } */

```

其中至少有五处问题：函数头末尾使用圆括号，函数体应以花括号开始；声明行 `int n, int n2, int n3` 重复关键字，合法写法为 `int n, n2, n3;`；注释只有 `/*` 而无配对的 `*/`，其后内容全部被当作注释；`printf` 语句缺分号；末尾以圆括号结束函数体。

关于语法错误，有三条经验：

- 编译器检查语法错误；
- 一处语法错误可能使编译器在其他位置也报告错误，报错数量常大于真实错误数量；
- 编译器报告的出错位置可能滞后于真实错误一行，应同时检查上一行。

据此，报错数量多并不代表真实错误数量多；修正一处错误往往能同时消除多条报错；报错行本身可能没有错误，问题出在上一行，比如缺了一个分号。编译器的报错是排查的起点，不是最终结论。

1.10.2 语义错误

语义错误指程序语义与设计意图不符。程序语法完全正确、可以运行并产生输出，但输出结果错误：

```

#include <stdio.h>
int main(void) {
    int n, n2, n3;
    /*the program has a semantic error*/

    n = 5;
    n2 = n*n;
    n3 = n2*n2;

```

```
printf("n=%d, n square = %d, n cubed = %d\n", n, n2, n3);  
return 0;  
}
```

程序正常运行，输出 `n cubed = 625`，而 $n = 5$ 的立方为 125。错误在 `n3 = n2*n2`： $n2$ 是 n 的平方， $n2 \times n2$ 是 n 的四次方，输出标签却为 “cubed”。编译器对此不做任何报告，因为每一行单独看都合法：编译器仅执行严格的语法分析，无法验证程序的语义逻辑是否符合开发者的预期。

语义错误的排查依靠人工手段：

- Step-by-Step 单步执行；
- 断点的设置；
- 变量值的跟踪；
- 在关键位置插入 `printf()` 监视特定变量。

单步与断点由调试器提供；插桩 `printf` 不依赖工具，任何环境可用。四种手段的目标一致：使“程序当前实际算出的值”可见，与“预期应有的值”对照。

1.10.3 第三种情况：代码之外

语法与语义的错误都在代码之内。上机时你们还会遇到一类报错，源头在代码之外。一个直观的类比：双击一个大型 3D 游戏，屏幕弹出“无法初始化 DirectX 11 渲染设备”。游戏本身没有写错，它是经过检验的商业产品，语法与逻辑都没有问题；问题出在它运行所依赖的环境：显卡驱动过旧、DirectX 组件缺失、硬件不支持。报错针对的不是代码，是代码脚下的软硬件上下文。

C 程序同样如此。双击 .exe 没有反应、提示缺少某个 .dll、程序在别人的机器上能跑而在你的机器上闪退，多半属于这一类：代码没错，是它依赖的运行库、操作系统版本或权限没有就位。这类问题在本课前期不作为重点，但有两件事要记在心上。其一，报错不等于代码写错，排障的头一步是分清错误在哪一层：编译期报错指向语法，运行结果不对指向语义，程序根本运行不起来则先查环境。其二，程序从不孤立运行，它依赖操作系统、库与硬件构成的完整上下文。这与本讲反复出现的“程序是存储器里的 0 和 1”并不矛盾：正是这套上下文把那串 0 和 1 装载、链接，并交到 CPU 手里。

1.10.4 改错练习

课件留有一道改错题。程序中的错误建议自行排查后再对照答案：

```
#include <studio.h>
int main{void} (
    int s
    S: = 56;

    printf(There are s weeks in a year.);
    return 0:
)
```

逐条列出：

1. `#include <studio.h>`: 头文件名拼写错误，应为 `stdio.h`。studio 为“工作室”，stdio 是 standard input/output 的缩写，两者仅两个字母顺序不同；
2. `int main{void} (`: 函数头参数表应使用圆括号 (`void`)，函数体应以花括号 { 开始，该行两个括号均用错；
3. `int s`: 声明缺分号；
4. `S: = 56;`: 一行含三个错误。大小写敏感，`S` 与已声明的 `s` 是两个标识符；C 的赋值号为单个 `=`，不存在 `:=`；`S` 后的冒号多余；
5. `printf(There are s weeks in a year.);`: 打印内容须置于双引号内；打印变量 `s` 的值应使用 `%d` 占位并以 `s` 作参数；
6. `return 0:`: 冒号应为分号；
7. `)`: 函数体应以花括号 `}` 结束。

修正后的程序：

```
#include <stdio.h>
int main(void) {
    int s;
    s = 56;

    printf("There are %d weeks in a year.\n", s);
    return 0;
}
```

语法修正之后仍有一个语义问题：`s = 56` 是否符合事实？一年约 52 周。数值正确性不在编译器的检查范围内，属于语义层面的审查。

1.11 本讲总结

概念层面。程序是对计算过程的精确描述，其存在不依赖具体机器：第一份程序写于可运行它的机器建成之前。编程语言与自然语言的分界在于格式必须完全正确，这一约束贯穿全课。

机器层面。计算机只处理 0 和 1，一切计算归结为位运算。二进制算术由莱布尼茨建立，布尔代数提供逻辑运算的代数工具，香农证明了其与开关电路的同构。开关器件从继电器、电子管演进到晶体管与集成电路。一位加法由异或门（Sum）与与门（Carry）构成的半加器实现；全加器串联为多位加法器，进位链构成延迟来源；加法器与控制信号结合扩展为 ALU，执行算术、逻辑、比较、移位四类运算。

架构层面。ENIAC 以接线编程的局限引出 1945 年的方案：将指令作为数据存入同一个存储器。存储程序计算机按取指、译码、执行、写回的指令周期逐条执行，每秒数十亿次，这是 GHz 主频的物理含义。CPU 的基本工作方式：数据须先进入寄存器，ALU 只操作寄存器，结果存回存储器。

语言层面。编程是把人类思维逻辑转化为机器可执行指令的过程，编译器完成翻译（霍珀 1952 年实现 A-0，巴科斯 1957 年以 FORTRAN 验证其可行性）。一行 C 语句对应多条汇编指令，变量名对应存储器地址，表达式对应寄存器操作；汇编再编码为以操作码打头的机器码。C 语言 1973 年完成于贝尔实验室，紧凑高效、可移植、可直接访问内存与硬件，内存管理的责任同时由程序员承担。C/C++ 与 Python 的分工（引擎与胶水）是芯片设计与验证工作流的现行范式。

程序层面。最小的 C 程序由预处理指令、main 函数、变量声明、语句和 return 组成。错误分三层来看：语法错误由编译器报告，报错位置可能滞后一行；语义错误须依靠单步、断点、变量跟踪与插桩 printf 排查；环境错误的源在代码之外，排障前先分清错误在哪一层。

参考文献

- [1] Abelson, H. and Sussman, G. J., *Structure and Interpretation of Computer Programs*, 2nd ed., MIT Press, 1996.
- [2] Backus, J., “The History of FORTRAN I, II, and III,” *IEEE Annals of the History of Computing*, vol. 1, no. 1, pp. 21–37, 1979.
- [3] Bezanson, J., Edelman, A., Karpinski, S., and Shah, V. B., “Julia: A Fresh Approach to Numerical Computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.
- [4] Boole, G., *An Investigation of the Laws of Thought*, Walton and Maberly, 1854.

-
- [5] Dijkstra, E. W., “Go To Statement Considered Harmful,” *Communications of the ACM*, vol. 11, no. 3, pp. 147–148, 1968.
- [6] Harris, D. M. and Harris, S. L., *Digital Design and Computer Architecture*, 2nd ed., Morgan Kaufmann, 2013.
- [7] ISO/IEC 9899:1999, *Programming Languages — C*, 1999.
- [8] Kernighan, B. W. and Ritchie, D. M., *The C Programming Language*, 2nd ed., Prentice Hall, 1988. (中译本《C 程序设计语言》)
- [9] Knuth, D. E., *The Art of Computer Programming*, Vol. 1: *Fundamental Algorithms*, 3rd ed., Addison-Wesley, 1997.
- [10] Leibniz, G. W., “Explication de l’Arithmétique Binaire,” *Mémoires de l’Académie Royale des Sciences*, 1703.
- [11] McCarthy, J., “Recursive Functions of Symbolic Expressions and Their Computation by Machine, Part I,” *Communications of the ACM*, vol. 3, no. 4, pp. 184–195, 1960.
- [12] Menabrea, L. F., “Sketch of the Analytical Engine Invented by Charles Babbage,” *Bibliothèque Universelle de Genève*, No. 82, 1842. 英译本附 Ada Lovelace 的注释 (Note G 含伯努利数程序), 1843.
- [13] Moore, G. E., “Cramming more components onto integrated circuits,” *Electronics*, vol. 38, no. 8, 1965.
- [14] Naur, P. (ed.), “Report on the Algorithmic Language ALGOL 60,” *Communications of the ACM*, vol. 3, no. 5, pp. 299–314, 1960.
- [15] Prata, S., *C++ Primer Plus*, 6th ed., Addison-Wesley, 2012. (中文版《C++ Primer Plus (第 6 版) 中文版》, 人民邮电出版社)
- [16] Ritchie, D. M., “The Development of the C Language,” *Proc. 2nd ACM SIGPLAN Conference on History of Programming Languages (HOPL-II)*, 1993.
- [17] Ritchie, D. M. and Thompson, K., “The UNIX Time-Sharing System,” *Communications of the ACM*, vol. 17, no. 7, pp. 365–375, 1974.
- [18] Shannon, C. E., “A Symbolic Analysis of Relay and Switching Circuits,” *Transactions of the AIEE*, vol. 57, no. 12, pp. 713–723, 1938.

- [19] Tanenbaum, A. S. and Austin, T., *Structured Computer Organization*, 6th ed., Pearson, 2013.
- [20] von Neumann, J., *First Draft of a Report on the EDVAC*, Moore School of Electrical Engineering, University of Pennsylvania, 1945.
- [21] Wirth, N., *Algorithms + Data Structures = Programs*, Prentice Hall, 1976.
- [22] 夏宽理, 陈子平, 《C 语言程序设计》(第三版), 中国铁道出版社.
- [23] 欧几里得, 《几何原本》, 约公元前 300 年(卷七记载最大公约数算法).
- [24] Hopper, G. M., “The Education of a Computer,” *Proc. 1952 ACM National Meeting (Pittsburgh)*, pp. 243–249, 1952.